

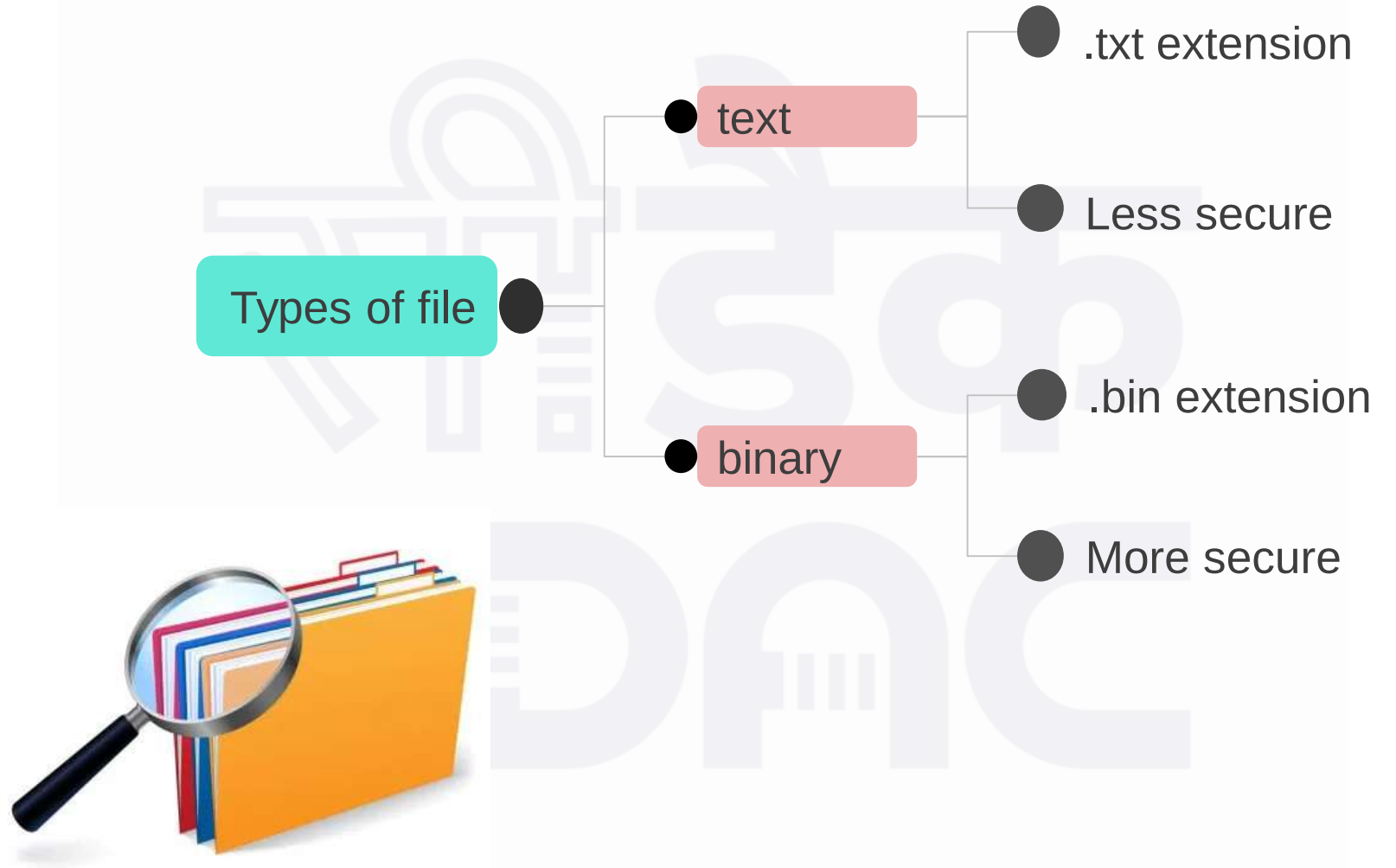
File Handling In C



Why we need file handling ?

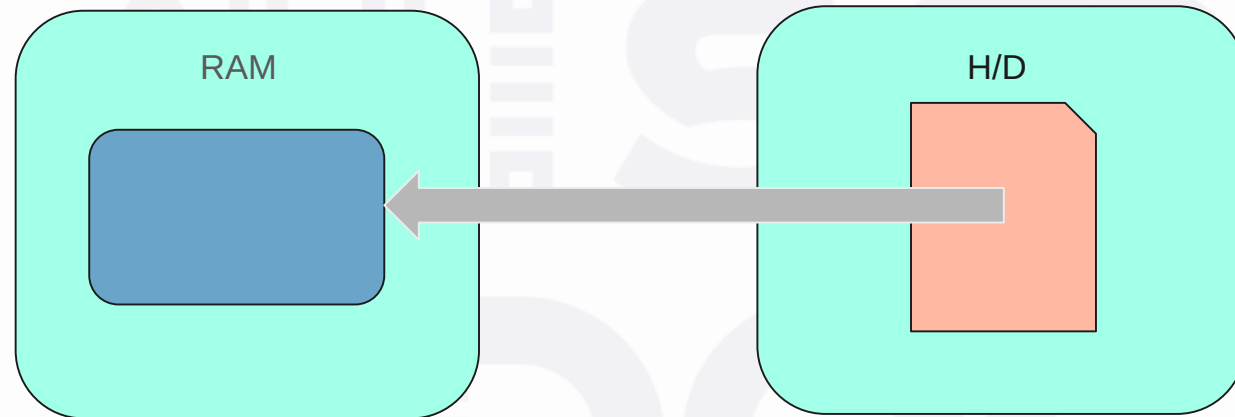
- To handle the file is called file handling.
- Memory : RAM , HARDDISK.
- **File** : sequence of bytes or some memory where data is stored permanently.
- After termination of program still you want to store the data.
- By few commands in C can access the file.
- **Example** : in program suppose we want to give large amount of data as input.
- To handle large amount of data for that we need file handling.

Types of File



How to access file?

- Cannot directly access file it has to be loaded in RAM memory.
- Need of pointer.



File Pointer

In order to access any file, a file pointer has to be declared as follows:

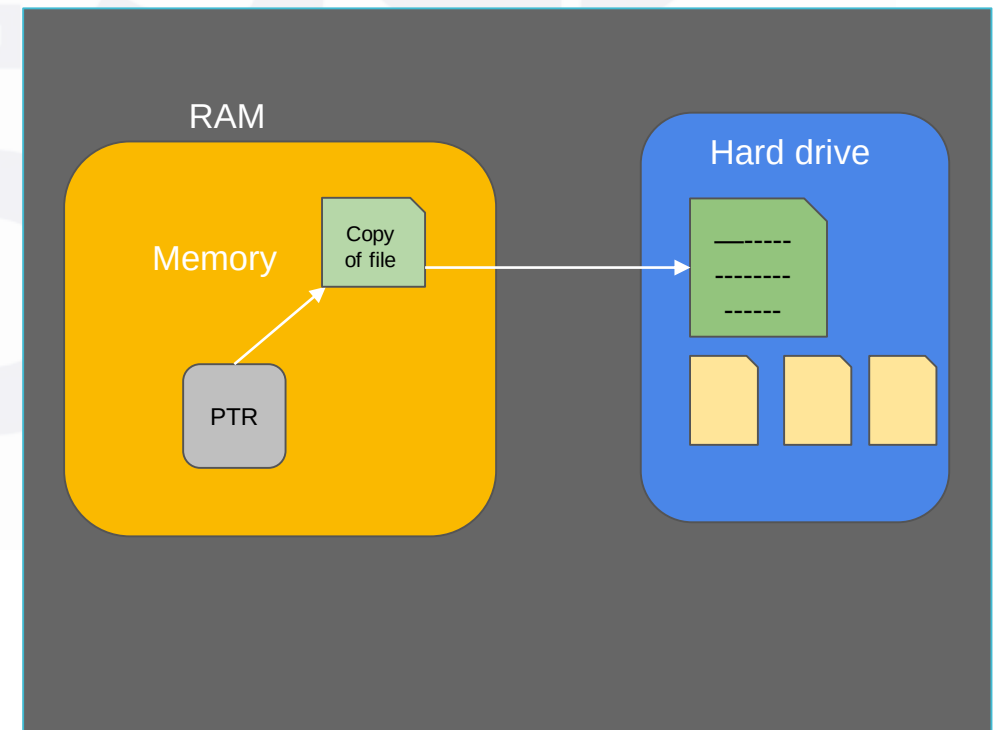
```
FILE *fileptr;
```

it will contain **the address of file type variable**.

A pointer which is going to point to the file that would be of file type.

FILE is data type defined in <stdio.h>.

```
FILE *ptr ; /Pointer points to that copied file.
```



fopen()

By calling fopen() it automatically creates file type of pointer.

It will return address of the file.

Load copy of file in buffer and ultimately return address and that address would be store in file pointer.

fileptr= fopen(“filename”,mode)

fclose(fileptr)

File Opening Modes

Mode	Meaning
r	Open a file for reading.
w	Open a file for writing. If the file exists, the <code>fopen()</code> function deletes all the former contents.
a	Open a file for appending. The <code>fopen()</code> function creates the file if it does not exist.
r+	It is for reading and writing but if file doesn't exist it will not create a new file
w+	Searches file. If the file exists, its contents are overwritten. If the file doesn't exist a new file is created. Returns NULL, if unable to open the file.
a+	Searches file. If the file is opened successfully <code>fopen()</code> loads it into memory and sets up a pointer that points to the last character in it. It opens the file in both reading and append mode. If the file doesn't exist, a new file is created. Returns NULL, if unable to open the file.

How to write a file in C (w mode)

"w" : Opens the file for writing.

"w" mode creates a new empty file if it doesn't exist.

Example: Writing to a File

```
#include <stdio.h>

int main() {
    FILE *fptr; // Declare a file pointer
    char data[] = "Hello,welcome to
learn C programming !";

    // Open the file "example.txt" in
write mode
    fptr = fopen("example.txt", "w");

    // Check if the file was opened
successfully
    if (fptr == NULL) {
        printf("Error: Could not open
the file.\n");
        return 1;
    }

    // Write the string to the file
    fprintf(fptr, "%s", data);

    // Close the file
    fclose(fptr);

    printf("Successfully wrote to
example.txt\n");

    return 0;
}
```

Read a text file (r mode)

r Mode Open a file for reading.

```
#include <stdio.h>

int main() {
    // Write to file

    FILE *file = NULL;

    // Read from file
    file = fopen("filename.txt","r");
    if (file == NULL) return 1;
    char ch = fgetc(file);
    printf("Read: %c\n", ch);
    fclose(file);
    return 0;
}
```

fgetc(file) reads exactly one character at a time from the file.

Always Check for NULL

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    // File pointer to store the value returned by fopen
    FILE *fptr;
    // Opening the file in read mode
    fptr = fopen("filename.txt", "r");
    // checking if the file is opened successfully
    if (fptr == NULL) {
        printf("The file is not opened.");
    }
    return 0;
}
```

Append mode (a) in file handling

If you want to add more
content in existing file.

```
#include <stdio.h>

int main() {
    FILE *fp = fopen("filename.txt", "a"); // Open in append mode

    if (fp == NULL) {
        printf("Cannot open file.\n");
        return 1;
    }

    fprintf(fp, "Appended line.\n"); // Adds this line to the end
    printf("successfully appended \n");
    fclose(fp);

    return 0;
}
```

r+ mode

- r+ mode is for both reading and writing .
- the main purpose is reading.
- If file exist it will open the file both for reading and writing(update) .
- its primary purpose is **reading**; critically, it will **NOT** create a new file if one doesn't exist.

w+ mode

- w+ mode is for both reading and writing .
- the main purpose is **writing**.
- If file exist it will truncate(erases) the existing content so u can add new content from the starting. Reading privileges added to it.
- If doesn't exist it will create a new file.

a+ mode

- **"a+"** mode stands for **"append and read"** mode.
- It opens a text file for both reading and writing (appending), ensuring that existing content is preserved and new data is always written at the end of the file.

Comparison

File Mode	Operations Allowed	Action If File Exists	Action If File Missing	Writing Position
"a+"	Read & Append	Preserves data	Creates new file	Always at the end
"r+"	Read & Write	Preserves data	Returns NULL (Error)	At the current pointer
"w+"	Read & Write	Overwrites / Truncates	Creates new file	At the current pointer

Renaming/ Deleting a File

The function **rename()** is used to rename a file. We will rename a file “example.txt” → “new.txt”.

```
#include <stdio.h>

int main()
{
    rename("example.txt", "new.txt");
}
```

The function **remove()** is used to delete a file. We will delete the file “new.txt”.

```
#include <stdio.h>
int main()
{
    remove("new.txt");
}
```

They return 0 on success and a non-zero value (usually -1) if an error occurs.

fputc()

- **fputc()** writes exactly **one single character** at a time to the destination file.
- Single characters in C must use **single quotes** ('H').
- C tracks where it is writing automatically. Every time fputc() runs, the internal file pointer advances by 1 space.
- fputc('\n', file); manually pushes the cursor down to the next line inside the text file.

fputs()

- **fputs()** stands for "file put string".
- It writes an entire block of text (string) all at once.
- Strings in C must use double quotes (").
- Unlike standard screen printing (puts()), fputs() does not automatically add a new line at the end of the text. You must explicitly include \n inside the string to break the line.

fprintf()

- **fprintf()** stands for "file print formatted".
- It works exactly like printf(), but instead of printing to your computer screen, it formats and outputs text directly into your file.
- **Example:**

```
fprintf(file, "Name: %s\n", name);  
fprintf(file, "Age: %d\n", age);  
fprintf(file, "Score: %.2f\n", score);
```
- It uses standard format specifiers to translate raw variables into text:
 - %s pulls the text string "Alice".
 - %d pulls the integer 30.
 - %.2f pulls the decimal number 95.6 and limits it to exactly 2 decimal places (95.60).

Comparing fputc(), fputs(), fprintf()

Output:

Hello
This is written using fputs.
Name: Alice
Age: 30
Score: 95.60

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *file =  
    fopen("output.txt", "w");
```

```
    if (file == NULL) {  
        printf("Error opening  
file");
```

```
        return 1;
```

```
    }
```

**// 1. Using fputc() - Writes
characters one at a time**

```
fputc('H', file);
```

```
fputc('e', file);
```

```
fputc('l', file);
```

```
fputc('l', file);
```

```
fputc('o', file);
```

```
fputc('\n', file); // Manually add newline
```

**// 2. Using fputs() - Writes a full string (no
formatting)**

```
fputs("This is written using fputs.\n", file);
```

// 3. Using fprintf() - Writes formatted strings

```
int age = 30;
```

```
double score = 95.6;
```

```
const char *name = "Alice";
```

```
fprintf(file, "Name: %s\n", name);
```

```
fprintf(file, "Age: %d\n", age);
```

```
fprintf(file, "Score: %.2f\n", score);
```

```
// Done writing
```

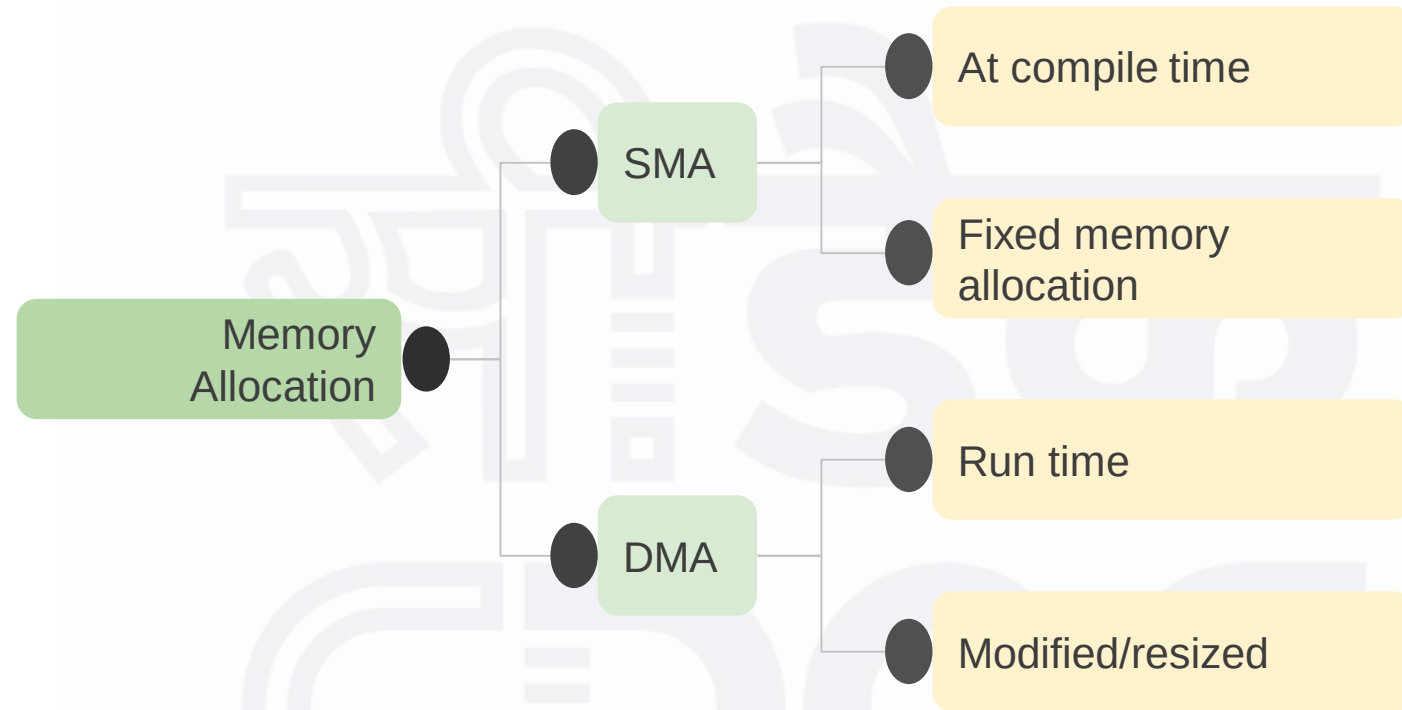
```
fclose(file);
```

```
printf("Data written to 'output.txt' using fputc,  
fputs, and fprintf.\n");
```

```
return 0; }
```

Memory Management

Static vs Dynamic Memory Allocation



Static Memory Allocation

- In SMA memory cannot be reused.
- In DMA we can allocate and deallocate memory at runtime. We have some functions in C: malloc , calloc, realloc and free.

STATIC MEMORY ALLOCATION

```
Int arr[10];
```

reserves exactly 10 integers' worth of memory at compile time. That size is locked in — it can never grow or shrink while the program runs.

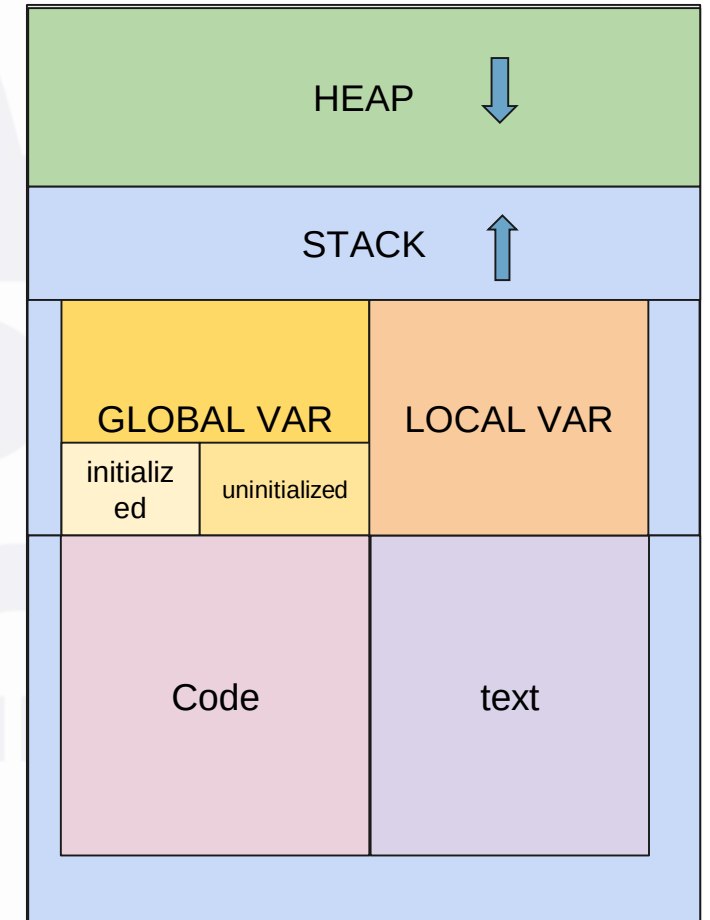
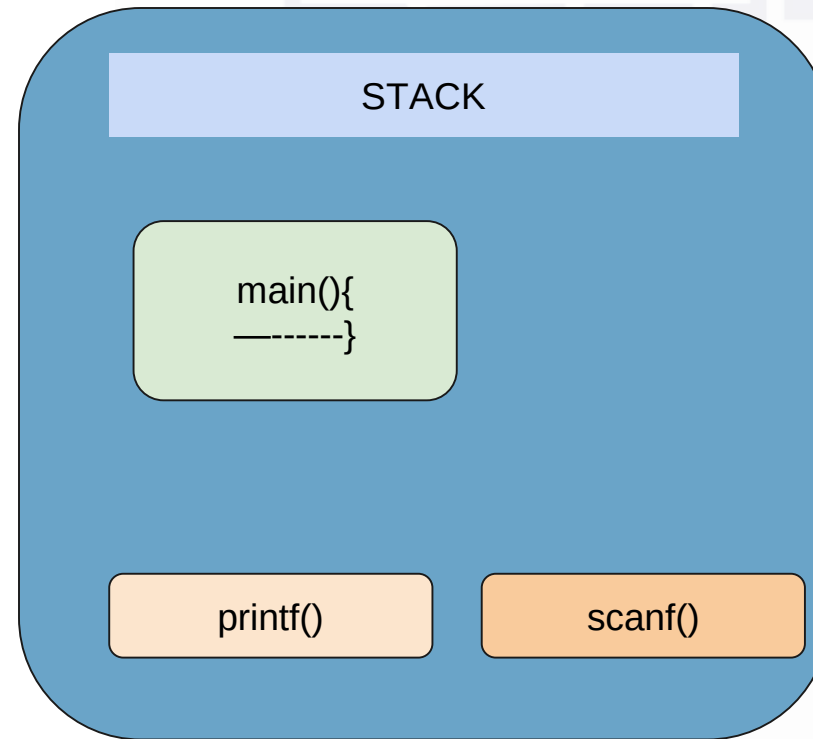


Static Memory Allocation (SMA) vs. Dynamic Memory Allocation (DMA)

Feature	Static Memory Allocation (SMA)	Dynamic Memory Allocation (DMA)
When it happens	Compile-time (before the program runs)	Runtime (while the program is running)
Memory location	Stack or Data Segment	Heap
Size flexibility	Fixed size; cannot be changed	Flexible size; can grow or shrink
Lifespan	Controlled by scope or program duration	Controlled manually by the programmer
Reusability	Cannot be manually freed or reused	Can be freed and reused immediately

Memory Layout of a C Program

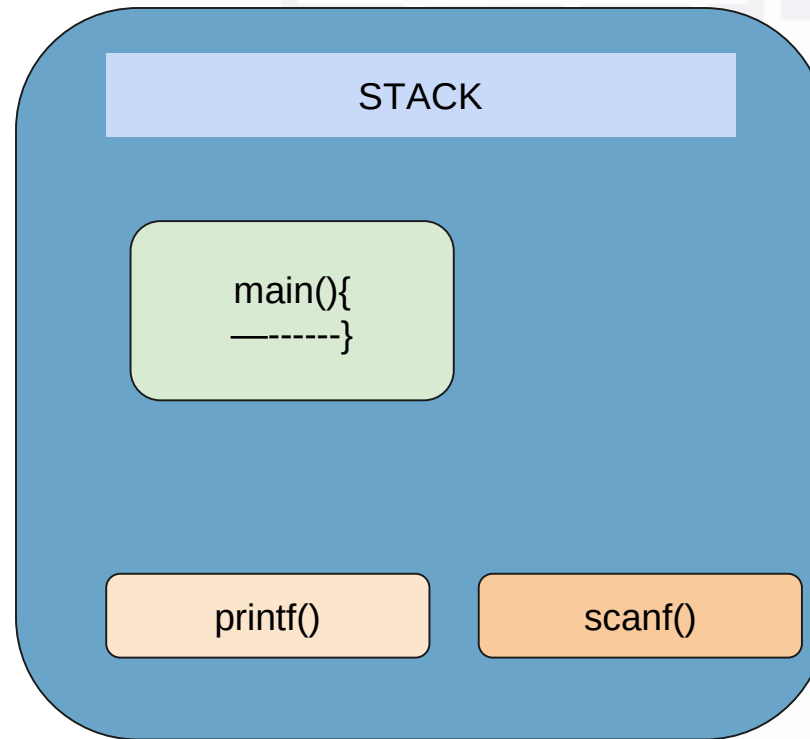
MEMORY LAYOUT 4 segments



Stack

⇒ Limited size.

⇒ All functions and variables take memory from stack .



Stack Memory vs. Heap Memory

Feature	Stack Memory	Heap Memory
Separation	Entirely separate segment	Entirely separate segment
Management	Managed automatically by the CPU	Managed manually by the programmer
Control	Handled through function calls	Handled using <code>malloc</code> , <code>calloc</code> , <code>free</code>
Growth direction	Grows toward the Heap	Grows toward the Stack
Access Speed	Extremely fast	Slower due to pointer lookups

Why DMA Needs a Pointer

Stack vs. Heap: Local variables automatically use Stack memory, which is freed instantly when a function ends.

No Names in the Heap: Dynamically allocated memory on the Heap has no variable name.

DMA requires a pointer to access heap memory

The Role of Pointers: A pointer is required to store the base address of the allocated block (e.g., an 8-byte block).

Manual Control: Because the system doesn't track heap names, you must use that pointer to manually free the memory when done.

The DMA Toolkit: malloc / calloc / realloc / free

The functions used to allocate memory:

malloc()-Allocates a block of memory but does not initialize it

calloc()-Allocates a block of memory and clears it (initializes the elements to 0).

realloc()-Resizes a previously allocated block of memory

free() - The functions used to deallocate memory.

All four live in **<stdlib.h>**.

Dynamic Allocation

When the size of a variable (string or array) is unknown before execution of a program, the size of the variable may be dynamically allocated.

Here, we will ask the user for the size of an integer array, and then we will allocate the amount of memory required using **malloc()** .

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n;
    printf("Enter the size of the array\n");
    scanf("%d", &n); //Enter size of array
    int *arr= malloc(n * sizeof(int)); //dynamic
    printf("Dynamic Allocation\n");
    free(arr); //do not forget to free the allocated memory
    return 0;
}
```

MALLOC and CALLOC

malloc stands for **Memory Allocation**.

It allocates a **block of memory** of a given size (in bytes) on the heap.

The content of the memory is **uninitialized** — it may contain **garbage values** (leftover data in memory).

```
Malloc (size);
```

What is **calloc()**?

calloc stands for **Contiguous Allocation**. It also allocates memory on the heap, but **initializes** all allocated bytes to **zero**.

DMA using malloc

```
ptr = (cast-type*) malloc(size_in_bytes);
```

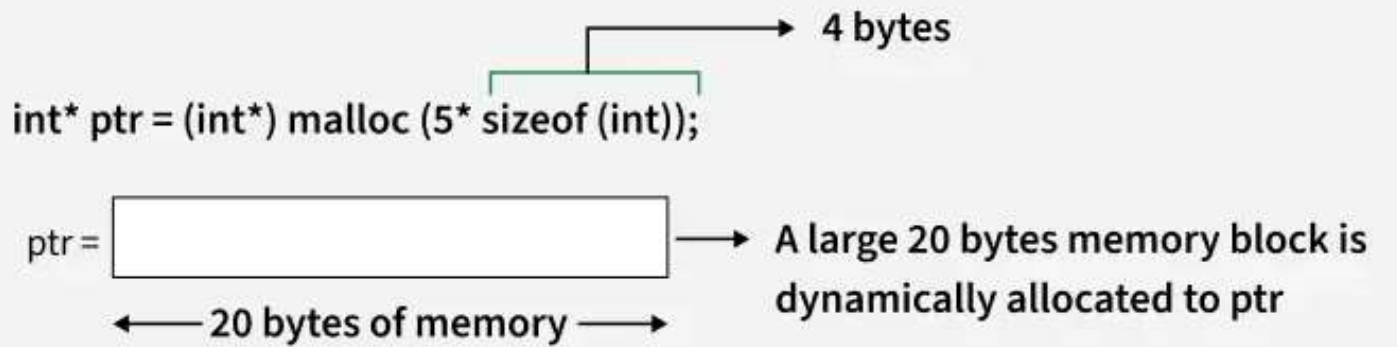
- `ptr` → The pointer variable that stores the starting address of the newly allocated heap memory.
- `(cast-type*)` → Converts the raw `void*` pointer returned by `malloc` into your specific data type (e.g., `int*` or `float*`).
- `malloc()` → The built-in memory allocation function.
- `size_in_bytes` → The total memory size requested (commonly calculated using `n * sizeof(data_type)`).

Practical Example of malloc()

```
int *p = (int*) malloc(5 * sizeof(int));
```

- Allocates memory for **5 integers**.
- `sizeof(int)` = number of bytes for one integer (usually 4 bytes).
- So total memory = $5 * 4 = 20$ bytes.
- `p` now points to the first integer's location of this 20-byte block.

Practical Example of malloc()



Example:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr;
    int n = 5; // number of integers

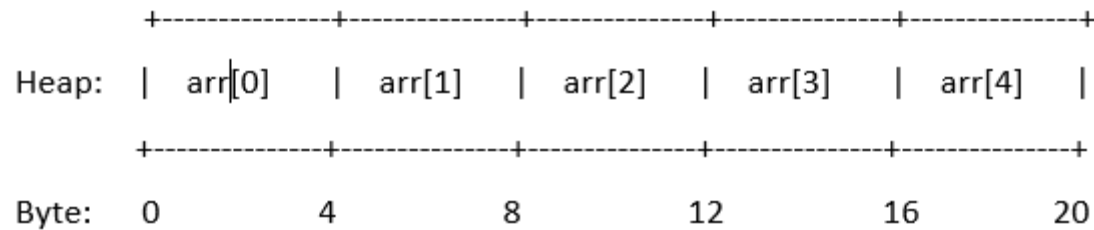
    // Allocate memory for 5 integers
    ptr = (int*) malloc(n *
sizeof(int));

    // Store values
    for (int i = 0; i < n; i++) {
        ptr[i] = i + 1;
    }

    // Print values
    printf("Using malloc:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", ptr[i]);
    }

    // Free memory
    free(ptr);

    return 0;
}
```



Example: malloc + scanf for Two Pointers

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
    int *a, *b;
```

```
    // Allocate memory for 'a' and 'b'
    a = (int*)malloc(sizeof(int));
    b = (int*)malloc(sizeof(int));
```

```
    // Check for successful memory
    allocation
    if (a == NULL || b == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }
```

```
    // Prompt the user to enter values
    printf("Enter a value for a: ");
    scanf("%d", a); // Store the input
    directly into the memory location a points
    to
```

```
    printf("Enter a value for b: ");
    scanf("%d", b); // Store the input
    directly into the memory location b points
    to
```

```
    // Print the values
```

```
    printf("You entered:\n");
    printf("Value of a: %d\n", *a);
    printf("Value of b: %d\n", *b);
```

```
    // Free the allocated memory
    free(a);
    free(b);
```

```
    return 0;
```

```
}
```

Output

```
Enter a value for a: 4
Enter a value for b: 5
You entered:
Value of a: 4
Value of b: 5
```

Calloc - Contiguous allocation

The calloc() function allocates memory and initializes all bits to zero.

Syntax: ptr = (castType*)calloc(n, size);

calloc(n, sizeof(int)) → allocates memory for n integers and initializes them to 0.

Void * calloc(noOfBlocks, sizeof(datatype))

Pointer will return the base address of the first

Example:

```
ptr = (float*) calloc(25, sizeof(float));
```

The above statement allocates contiguous space in memory for 25 elements of type float.

Calloc - Contiguous allocation

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    // Allocate memory for 3 integers (two numbers and their sum)
    // The memory is automatically initialized to 0 by calloc.
    int *numbers = (int*) calloc(3, sizeof(int));
    // Check if memory allocation was successful
    if (numbers == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }
    // Assign values to the first two allocated memory locations
    numbers[0] = 5;
    numbers[1] = 10;
    // Calculate the sum and store it in the third allocated memory location
    numbers[2] = numbers[0] + numbers[1];
    // Print the result from the allocated memory
    printf("The sum of %d and %d is %d.\n", numbers[0], numbers[1], numbers[2]);
    // Free the dynamically allocated memory
    free(numbers);
    return 0; }
```

Realloc

If the dynamically allocated memory is insufficient or more than required, you can **change the size of previously allocated memory** using the **realloc()** function.

Reallocating reserves a different (usually larger) amount of memory while keeping the data that was stored in it.

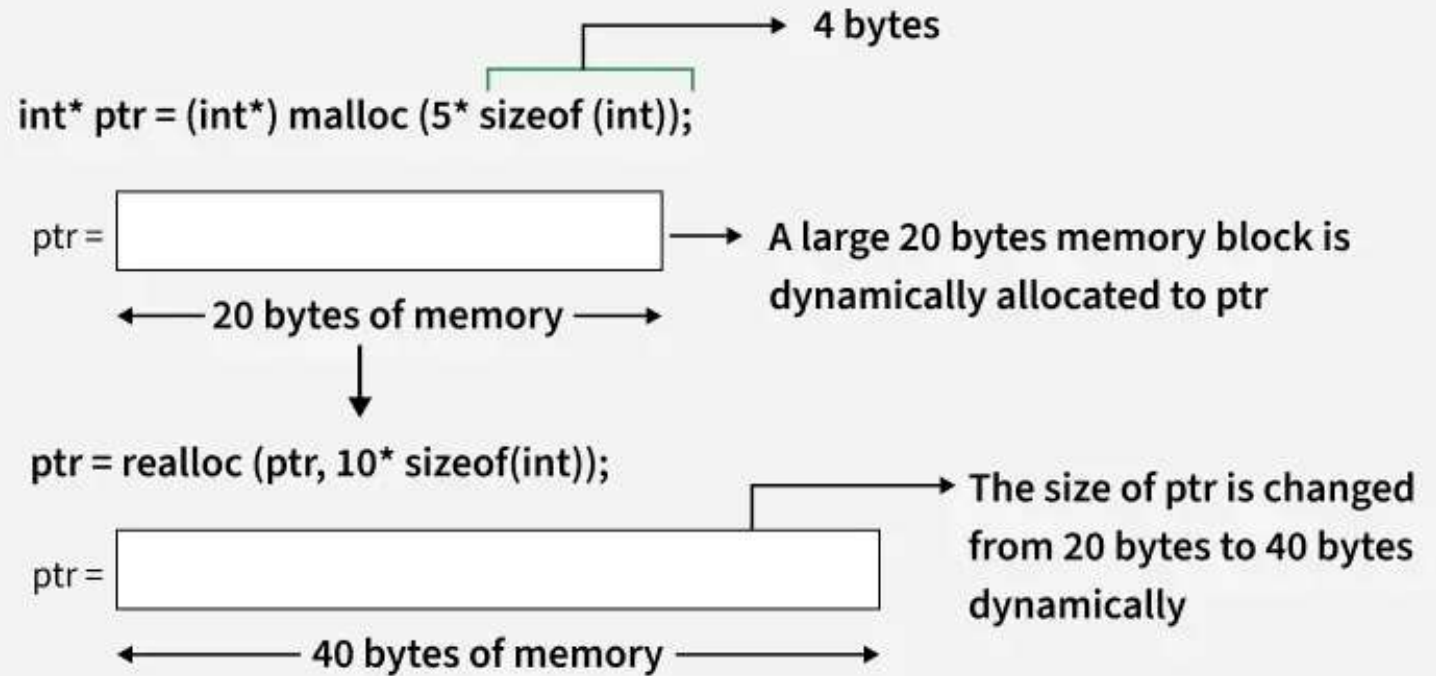
The realloc() function takes two parameters:

Syntax: `ptr = realloc(ptr, x);`

Here, ptr is reallocated with a new size x.

```
int *ptr2 = realloc(ptr1, size);
```

Realloc



Realloc

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr;
    int *ptr2;
    int n = 3;

    ptr = (int*) malloc(n *
sizeof(int));

    for (int i = 0; i < n; i++) {
        ptr[i] = i + 1;
    }

    printf("\n\nBefore
realloc:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", ptr[i]);
    }

    // Increase size to 5
    n = 5;
    ptr2 = (int*) realloc(ptr, n * sizeof(int));

    // Add new values
    ptr2[3] = 4;
    ptr2[4] = 5;

    printf("\nAfter realloc:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", ptr2[i]);
    }

    free(ptr2);

    return 0;
}
```

Output

Before realloc:

1 2 3

After realloc:

1 2 3 4 5

=== Code Execution Successful ===

C Enumeration (enum)

C Enumeration (enum)

An enumeration (enum) in C is a user-defined data type used to assign names to integral constants. They make a program easier to read and maintain by replacing "magic numbers" (like 0, 1, 2) with descriptive names (like LOW, MEDIUM, HIGH). An enum is a special type that represents a group of constants (unchangeable values).

The basic syntax for defining an enum is: Use the **enum** keyword followed by the type name and a list of constants inside curly braces.

```
enum tag_name {  
    constant1,  
    constant2,  
    constant3,  
    // ...  
};  
  
enum Level {  
    LOW,    // assigned 0 by default  
    MEDIUM, // assigned 1 by default  
    HIGH   // assigned 2 by default  
};
```

Access enums in C

```
enum Level {  
    LOW,  
    MEDIUM,  
    HIGH  
};  
enum Level myVar;
```

```
#include <stdio.h>  
#include <stdlib.h>  
  
enum Level {  
    LOW = 1,  
    MEDIUM,  
    HIGH  
};  
  
int main() {  
    enum Level myVar = MEDIUM;  
  
    switch (myVar) {  
        case 1:  
            printf("Low Level");  
            break;  
        case 2:  
            printf("Medium level");  
            break;  
        case 3:  
            printf("High level");  
            break;  
    }  
    return 0;  
}
```

Example: Days of the Week

```
enum Weekday  
{ SUNDAY,  
  MONDAY,  
  TUESDAY,  
  WEDNESDAY,  
  THURSDAY,  
  FRIDAY,  
  SATURDAY };
```

Assigning Custom Values

You can also explicitly assign values to the enumerators. This is useful if you need to skip numbers or assign a specific value.

```
enum Status { SUCCESS = 1, ERROR = -1, PENDING = 0  
};
```

If you don't assign a value to an enumerator, it gets the value of the previous enumerator plus one.

For example, in the Weekday enum, if you set MONDAY = 1, TUESDAY would automatically be 2.

Common Use Cases for Enums

Menu Options: Instead of using arbitrary numeric selections (like 1 for New Game, 2 for Load), enums allow you to **explicitly write text actions** like `NEW_GAME` or `QUIT` directly in your code logic.

Error Codes: Replaces hard-to-remember system codes (like error -1 or 404) with clear, self-explanatory markers such as `FILE_NOT_FOUND` or `PERMISSION_DENIED`.

State Machines: Perfect for **objects that can only occupy one specific mode at any given moment**. For example, a media player can only be strictly `PLAYING`, `PAUSED`, or `STOPPED`.

Flag Management: Uses powers of two (1, 2, 4, 8...) **combined with bitwise operators to merge multiple configuration settings** together into a single numeric variable (e.g., enabling both `COLOR` and `DOUBLE_SIDED` printing at the same time).

Summary

- enum makes your code more readable and less prone to errors.
- It's a collection of named integer constants.
- By default, values start from 0, but you can assign custom values.
- It's a powerful tool for improving the clarity and maintenance of your C programs.

Example: Weekday check with if

```
#include <stdio.h>

// Define the enumeration
enum Weekday {
    SUNDAY,
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
    SATURDAY
};

int main() {
    // Declare a variable of type Weekday
    enum Weekday today;

    // Assign a value to the variable
    today = MONDAY;

    // Use the enum in a conditional statement
    if (today == MONDAY) {
        printf("It's the start of the work week!\n");
    }

    // Print the integer value of the enum
    constant
    printf("The integer value of MONDAY is:
    %d\n", today);

    return 0;
}
```

Example 2: Traffic Light — switch Setup

Create a C program that prompts the user for a number representing a traffic light state. The program should use an enumeration to interpret the input and then print a corresponding action for a driver.

```
case RED:
    printf("The light is RED.
Please STOP!\n");
    break;
case YELLOW:
    printf("The light is
YELLOW. Prepare to STOP.\n");
    break;
case GREEN:
    printf("The light is GREEN.
You can GO.\n");
    break;
default:
    printf("Invalid light state
entered.\n");
    break;
```

Example 3: Traffic Light — Full Program

```
#include <stdio.h>
// Define the enumeration for traffic
light states
enum TrafficLightState {
    RED,    // This corresponds to
integer 0
    YELLOW, // This corresponds to
integer 1
    GREEN   // This corresponds to
integer 2
};
int main() {
    // Declare a variable of type
TrafficLightState
    enum TrafficLightState
currentLight;
    // Prompt the user for input
    printf("Enter the state of the traffic
light (0 for RED, 1 for YELLOW, 2 for
GREEN): ");
    // Read an integer from the user
and assign it to the enum variable
    scanf("%d", &currentLight);

    // Use a switch statement to check the
current state
    switch (currentLight) {
        case RED:
            printf("The light is RED. Please
STOP!\n");
            break;
        case YELLOW:
            printf("The light is YELLOW.
Prepare to STOP.\n");
            break;
        case GREEN:
            printf("The light is GREEN. You
can GO.\n");
            break;
        default:
            printf("Invalid light state
entered.\n");
            break; }

    // Print the integer value of the enum
constant
    printf("The integer value of the entered
state is: %d\n", currentLight);
    return 0; }
```

Union In C

Union

A union is a user-defined data type in C, similar to a struct.

The key difference is that all members of a union share the **same memory location**.

The size of a union is equal to the size of its largest member.

It's a way to store different data types in the same memory space, but only one at a time.

Union vs Struct

Memory Comparison: struct vs union

Imagine you have a char (1 byte) and an int (4 bytes).

struct (Separate memory): Allocates $(1 + 4 = 5)$ bytes.

You can use both variables simultaneously.

union (Shared memory): Allocates only 4 bytes (the size of the int). Both variables point to the exact same starting address.

Union Declaration

```
union union_name{  
    type1 member1;  
    type2 member2;  
    type3 member3;  
    .  
    .  
}
```

Initialization and Accessing

```
union Student {  
    int rollNo;  
    float height;  
    char firstLetter;  
};  
  
// Declare a union variable  
  
union Student data;  
  
// Assign and print the roll number  
  
data.rollNo = 21;
```

Initialization and Accessing – example program

```
#include <stdio.h>
```

```
// Define a union with  
// different data types  
union Student {  
    int rollNo;  
    float height;  
    char firstLetter;  
};
```

```
int main() {  
    // Declare a union variable  
    union Student data;  
  
    // Assign and print the roll number  
    data.rollNo = 21;  
    printf("%d\n", data.rollNo);  
    data.height = 5.2;  
    printf("%.2f\n", data.height);  
    data.firstLetter = 'N';  
    printf("%c", data.firstLetter);
```

```
    return 0;
```

```
}
```

Size of Union

size of the largest
member of the union

```
#include <stdio.h>
```

```
union A{  
    int x;  
    char y;  
};
```

```
union B{  
    int arr[10];  
    char y;  
};  
int main() {
```

```
    // Finding size using sizeof() operator  
    printf("Sizeof A: %ld\n", sizeof(union A));  
    printf("Sizeof B: %ld\n", sizeof(union B));  
    return 0; }
```

Union: Key Points

- Union members share the same memory location.
- The size of a union is determined by its largest member.
- They are used for memory optimization.
- You can only use one member of a union at a time.

Struct vs Union — Comparison

Feature	struct	union
Memory Allocation	Each member is allocated a separate memory space. The total size is the sum of all members' sizes (plus any padding).	All members share the same memory location. The size is equal to the size of the largest member.
Purpose	To group different but related data types together. You can use all members at the same time.	To save memory by storing different data types in the same space. You can only use one member at a time.
Member Access	Accessing one member does not affect the others. All members can hold valid data simultaneously.	Assigning a value to one member overwrites the value of any other member. Only the last assigned member's value is valid.

MCQs Memory



1. Which of the following is not a valid function for memory allocation:

a) malloc()

b) ralloc()

c) realloc

d) calloc

b) ralloc()



2. Which library is used for memory allocation operation?

- a) <stdio.h>
- b) <iostream>
- c) <stdlib.h>
- d) <string.h>

c) <stdlib.h>



3. Which type of memory allocation is used when the size to be allocated is not known beforehand.

- a) Automatic
- b) Dynamic
- c) Variable
- d) Static

b) Dynamic



4. Which of the following is not an advantage of dynamic memory allocation?

- a) Allows allocation at runtime
 - b) Uses only the required amount of memory
 - c) Uses less computing power
 - d) Allocated memory persists until freed
- c) Uses less computing power



5. Any program that leaves garbage is said to have

a) Underflow

b) Overflow

c) Segmentation fault

d) Memory leak

d) Memory leak

MCQs File



1. The header file required to use file input/output operations:

- a) <stdio.h>
- b) <stdlib.h>
- c) <iostream>
- d) <string.h>

a) <stdio.h>



2. Correct declaration of a pointer “fp”

a) FILE fp*;

b) file fp*;

c) file* fp;

d) FILE *fp;

d) FILE *fp;



3. Which of the following is an invalid mode for file operation?

a) 'a'

b) 'w'

c) 'c'

d) 'r'

c) 'c'



4. If the specified file does not exist, fopen() returns:

a) 0

b) 1

c) NULL

d) -1

c) NULL



5. File operations can be performed only on text (.txt) files and code(.c, .java, etc.) files

a) TRUE

b) FALSE

b) FALSE